Machine Learning with Applications 22 (2025) 100737

Contents lists available at ScienceDirect

Machine Learning with Applications

journal homepage: www.elsevier.com/locate/mlwa

Multi-source plume tracing via multi-agent reinforcement learning under common UAV-faults

Pedro Antonio Alarcon Granadeno *, Theodore Chambers, Jane Cleland-Huang

University of Notre Dame, Computer Science and Engineering, 384 Fitzpatrick Hall of Engineering, Notre Dame, 46556, IN, USA

ARTICLE INFO

Keywords:
Multi-agent reinforcement learning (MARL)
Fault-tolerant UAV systems
Small Unmanned Aerial Systems (sUAS)
Action-conditioned recurrent networks
Partially observable markov game (POMG)
Long short-term memory (LSTM)
Action-Specific Double Deep Recurrent Q-Network (ADDRQN)
Gaussian Plume Dispersion Model (GPM)

ABSTRACT

Hazardous airborne gas releases from accidents, leaks, or wildfires require rapid localization of emission sources under uncertain and turbulent conditions. Traditional gradient-based or biologically inspired strategies struggle in multi-source environments where odor cues are intermittent, aliased, and partially observed. We address this challenge by formulating multi-source plume tracing in three-dimensional fields as a cooperative partially observable Markov game. To solve it, we introduce an Action-Specific Double Deep Recurrent Q-Network (ADDRQN) that conditions on action–observation pairs to improve latent-state inference, and integrates teammate information through a permutation-invariant set encoder. Training follows a randomized centralized-training and decentralized-execution regime with host randomization, team-size variation, and noise injection. This yields a policy that is robust to agent failures (hardware malfunction, battery depletion, etc.), resilient to intermittent communication blackouts, and tolerant of sensor noise. Empirical evaluation in simulated Gaussian plume environments shows that ADDRQN achieves higher success rates and shorter localization times than non-action baselines, maintains strong performance under mid-mission disruptions, and scales predictably with team size.

1. Introduction

Airborne releases of hazardous gases from industrial accidents, pipeline leaks, and wildfires threaten public health and infrastructure. Effective response hinges on fast localization of one or more emitting sources despite uncertain, time-varying winds and turbulence. Atmospheric dispersion at operational scales is often modeled with Gaussian plume–based approaches such as AERMOD, which remain standard for short-to-medium range prediction and regulation; these models clarify how advection–diffusion and stability class shape the time-averaged concentration field and provide structural cues (e.g. mean wind, vertical stratification, and plume spread) that can inform search policies (Stockie, 2011; U.S. Environmental Protection Agency, 2024).

A substantial body of work in robotics and field sensing has tackled plume tracing through gradient-based, bio-inspired, information-theoretic, and probabilistic formulations. Gradient-based methods treat plume tracing as moving uphill in a concentration field; examples range from pseudo-gradient controllers that fuse anemotaxis and chemotaxis on micro-UAVs (Neumann et al., 2013) to conjugate-direction line minimizations (Mayhew et al., 2007) and Bayesian-aided gradient schemes (Yungaicela-Naula et al., 2018). These approaches are appealing for their simplicity, but in turbulent flows odor arrives in intermittent patches rather than smooth slopes, so instantaneous gradients often vanish or point the wrong way, which makes gradient-based controllers unreliable (Francis et al., 2022). Biologically inspired strategies emulate cast-and-surge or patterned searches (e.g., moth-like zigzags or outward spirals) and have shown utility in specific regimes (Alvear et al., 2018; Clark & Fierro, 2005; Hayes et al., 2002; Li et al., 2006). Yet their hand-crafted behaviors can be rigid, oversample space, and degrade when wind shifts or obstacles disrupt the assumed pattern. Information-theoretic and probabilistic methods address uncertainty more explicitly; for example, infotaxis maximizes expected information gain from sporadic hits (Vergassola et al., 2007), and filtering or mapping pipelines infer source location from noisy measurements, but they often optimize surrogate criteria rather than time-to-localize. In emergency response, time-to-localize is the primary performance objective, thus optimizing surrogate criteria (e.g., entropy reduction or coverage) can inadvertently prolong searches even when maps improve (Song et al., 2019).

Unlike gradient-based, bio-inspired, and information-theoretic schemes, RL does not require smooth gradients, predefined patterns, or coverage proxies and has recently emerged as a viable alternative to fast gas localization. For instance, Chen et al. showed a DQN that outperformed two classical odor-localization baselines in a simulated single-source indoor plume (Chen et al., 2021); Singh et al. trained

* Corresponding author.
E-mail addresses: palarcon@nd.edu (P.A. Alarcon Granadeno), tchambe2@nd.edu (T. Chambers), janeClelandHuang@nd.edu (J. Cleland-Huang).

https://doi.org/10.1016/j.mlwa.2025.100737
Received 19 April 2024; Received in revised form 20 August 2025; Accepted 12 September 2025
Available online 18 September 2025

recurrent agents whose emergent policies resembled cast-and-surge insect strategies and whose memory handled intermittent cues (Singh et al., 2023); and Loisy and Heinonen benchmarked a deep-RL solver that competes with state-of-the-art POMDP methods while remaining lightweight for real-time execution (Loisy & Heinonen, 2023).

Despite this progress, existing RL-based plume tracing studies are nonetheless plagued with their own set of challenges that motivate our approach. First, most RL studies use single-robot settings; to our knowledge there is no work that jointly trains multiple agents to cooperatively localize odor sources. In emergency response, this limits the efficiency gains that can be achieved via concurrent search. Second, the case of multiple simultaneous plumes (e.g. several leaking sources in the same area) remains largely unaddressed in the learning-based literature and most research assumes one source at a time. The multi-source scenario introduces additional complexity because overlapping plumes create ambiguous signals; an agent may detect odor but struggle to discern which source it originated from if all plumes carry the same chemical signature. This ambiguity can confound traditional single-source strategies. Third, many of the aforementioned algorithms setups restrict the search to a plane or fix the flight altitude. While appropriate for UGVs that operate on a plane, this restriction limits UAVs, which can exploit vertical structure. As Francis et al. notes, imposing a fixed minimum flight altitude can be suboptimal when source height or gas buoyancy varies; for buoyant releases, an incorrect altitude causes a UAV to dip in and out of the plume and misdeclare the source (Francis et al., 2022). In RL, moving from 2D to 3D adds an additional challenge: the state/observation space grows roughly exponentially with each added dimension under typical discretizations, which drives up sample complexity, training time, and the need for larger models. Lastly, and most important for field use, real teams operate under partial observability, sensor noise, unreliable communications, and platform faults. Gielis et al. document message loss, latency, and out-of-order delivery in multi-robot networks and note that many coordination algorithms are still evaluated under idealized, lossless links (Gielis et al., 2022). Petritoli et al. estimate UAV failure rates on the order of 10^{-3} per flight hour (about two orders higher than crewed aviation) and report frequent sensor and communications degradations (Petritoli et al., 2018). As a result, policies must remain effective as teammates drop in and out and as links degrade, yet most gas-localization evaluations continue to assume reliable communications and fault-free operation (Gielis et al., 2022).

This paper addresses these challenges with a host-invariant, action-specific Double Deep Recurrent Q-Network (ADDRQN) for cooperative multi-UAV plume tracing in **3D multi-source** settings and is robust to common multi-UAV faults. At the heart of our approach is a host-invariant backbone that attenuates partial observability and sensor noise by conditioning on action-observation pairs; because gas measurements are path dependent and exposure is control dependent, concatenating the previous action with the current observation improves latent-state inference and reduces aliasing between geometry-induced zeros and true absence of plume. To train this backbone, we adopt centralized training with decentralized execution (CTDE) and introduce two mechanisms, namely, random host assignment and randomized team sizes, which are designed to promote role invariance and robustness to common multi-UAV faults. First, we share weights across agents so a single parameter set defines behavior for all. During training, we alternate which agent is designated as the host when computing targets and gradients. As a results any agent can assume the host role at test time without retraining. Second, randomized team size exposes the learner to variable-cardinality interactions: a permutation-invariant set encoder summarizes teammates so the network accepts any number of collaborators, and during updates the model is trained with teammate-history inputs of differing lengths. This enables the policy to operate with arbitrary team sizes at deployment and remain reliable when teammates malfunction during mission. Therefore, our contributions are as follows:

1. We formulate multi-source plume tracing as a cooperative MARL problem with partial observability and propose an ADDRQN (Fig. 3), a host-invariant, action-specific recurrent value function that fuses host and teammate action–observation histories while using double-Q targets for stability.
2. We introduce team-size-randomized CTDE training regime with bucketed replay buffers, host randomization and noise injection. A training technique to yield policies that are robust to common multi-agent faults, namely, UAV drop-in/drop-out, intermittent communications and noisy sensors.
3. We conduct a large ablation over network widths/depths and module choices to isolate the effect of action-specific embeddings versus non-action counterparts.

To the best of our knowledge, this is the first application of cooperative MARL to plume localization. We treat robustness to common faults as a first-class design objective through a host-invariant recurrent value function and a randomized CTDE regime.

2. Related work

Reinforcement learning has recently been applied to robotic plume tracing, but prior work has largely focused on single-agent scenarios, single-source settings, and 2D environments. For example, Chen et al. use a DQN that ingests stacked sensor histories to cope with partial observability and demonstrate improvements over classical baselines (Chen et al., 2021). Zhao et al. propose PC-DQN, which clusters odor particles to enrich feature representations before value learning, achieving strong performance in turbulent single-agent simulations (Zhao et al., 2022). Hu et al. design a DDPG controller with an LSTM module for AUV plume following under dynamic flow and uses history to stabilize decisions (Hu et al., 2019). Loisy and Heinonen benchmark deep RL on the olfactory-search POMDP and demonstrates it performs competitively with approximate POMDP solvers while remaining computationally lightweight (Loisy & Heinonen, 2023). Singh et al. train recurrent agents that learn insect-inspired, history-dependent strategies for plume tracking in turbulent flows (Singh et al., 2023).

Collectively, these studies validate RL for odor localization, but they do not address cooperative multi-agent policies, simultaneous multi-source fields, or full 3D search. Our study tackles a strictly more challenging regime: multi-source plume tracing in 3D with collaborating agents. Multi-source environments introduce *signal aliasing* when chemically indistinct plumes overlap, confounding strategies that rely solely on local cues (e.g., gradient ascent). Additionally, extending RL from 2D to 3D expands the observation and action spaces combinatorially in grid-based or discretized domains, substantially increasing sample complexity, training time, and model capacity requirements; pressures avoided in prior 2D, single-robot studies. In practice, capacity and hyperparameter settings chosen for 3D tend to transfer down to 2D without change, but the converse is not true. Finally, real multi-robot deployments must contend with partial observability, sensor noise, communication failures, and agent dropout (Gielis et al., 2022; Petritoli et al., 2018). Policies trained without such variability often exhibit brittle behavior at test time. Our approach explicitly incorporates these real-world factors into both model design and training.

Outside plume tracing literature, our approach is most directly related to two recent research fronts. First, Everett et al. introduced GA3C-CADRL for multi-agent collision avoidance. The central architectural idea is to use an LSTM as a set encoder to handle a variable number of neighboring agents by processing each neighbor's state sequentially and extracting the final hidden state as a fixed-length embedding. Training follows the standard A3C/GA3C paradigm, where many simulator instances run in parallel, generate experiences and feeds them concurrently to a shared actor–critic network. While the GA3C introduces parallelism to improve training speeds, its asynchronous

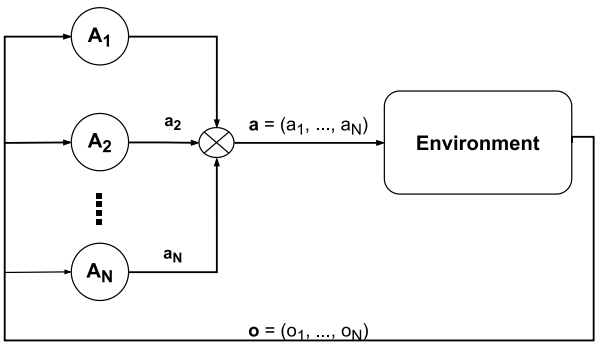


Fig. 1. Structure of a Partially Observable Markov Game (POMG). Individual decisions from agents contribute to a joint action vector **a** which influences the shared environment. This results in a joint observation vector and an immediate global reward for the agents.

updates are well documented to suffer from "policy lag" created by queued experience and predictions, which can hurt learning stability and sample efficiency (Babaeizadeh et al., 2017). In fact, Everett et al. relied on a supervised pre-initialization on CADRL-generated state–action-value pairs, followed by staged two phases of RL fine tuning. Our approach has two main differences.

Our approach has two main differences. First, we avoid the instability of asynchronous learning by adopting a DRQN-style, off-policy value-based training regime with centralized training and decentralized execution. During exploration, we vary the number of agents per episode and store transitions in replay buffers stratified by team size. During training, by first selecting a team size and then assigning a random host index to construct inputs for which targets and losses are computed. This enforces host-invariance and enables the shared policy to generalize across variable team sizes while avoiding GA3C instability. Second, whereas GA3C-CADRL uses an LSTM to embed only observations, we extend this module to consume observation–action pairs. Specifically, we concatenate each agent's previous action with its current observation and integrate the resulting sequence with a GRU-based encoder. This improves latent belief inference in partially observable settings where measurements are intermittent and control-dependent, as is the case in plume tracing. In contrast, collision-avoidance tasks operate under stronger observability assumptions (e.g. goal positions are specified).

This leads to our second closest connection, ADRQN by Zhu et al. which conditions the recurrent backbone on observation–action pairs to improve belief inference in single-agent partially observable environments. We adopt this principle and extend it to the multi-agent setting by embedding each agent's previous action with its current observation into a shared-weight recurrent Q-network. Furthermore, unlike in Zhu et al. (2018), we incorporate double-Q targets to reduce overestimation and train under a CTDE-style regime as described earlier.

3. Background

Multi-agent plume tracing operates under several central challenges: partial observability, the need for coordinated decision-making across multiple UAVs, and the requirement to tolerate UAV faults such as hardware failures, intermittent communications, and noisy sensors. We capture these aspects by modeling our problem as a Partially Observable Markov Game (POMG) (Puterman, 1990), formally defined by the eight-tuple $\langle N, S, \{A_i\}, P, R, \gamma, \{O_i\}, Z\rangle$, where N denotes the number of agents, S the global state space, and $\{A_i\}$ the collection of each agent's action sets whose Cartesian product $\mathcal{A} = A_1 \times \cdots \times A_N$ forms the joint action space. The transition kernel $P(s' \mid s, \mathbf{a}) : S \times \mathcal{A} \times S \to [0,1]$ specifies the probability of reaching state s' from s under joint action $\mathbf{a}$. The shared reward mapping $R(s, \mathbf{a}, s') : S \times \mathcal{A} \times S \to \mathbb{R}$ quantifies the immediate payoff, while the discount factor $\gamma \in [0,1)$ balances the weight of future versus immediate returns. Partial observability enters through per-agent observation sets $\{O_i\}$ and the function $Z(s, \mathbf{a}, \mathbf{o}) : S \times \mathcal{A} \times (O_1 \times \cdots \times O_N) \to [0,1]$, which gives the likelihood of joint observation $\mathbf{o} = (o_1, \ldots, o_N)$ when the environment transitions under $(s, \mathbf{a})$.

Interaction unfolds in discrete steps: at time t, agent i receives observation o_t^i, selects action a_t^i according to its policy $\pi(a \mid h_t^i)$. The collective actions of N agents forms a joint action $\mathbf{a}_t = (a_t^1, \ldots, a_t^N)$ which triggers a transition $s_{t+1} \sim P(\cdot \mid s_t, \mathbf{a}_t)$. The environment then emits a subsequent observation $\mathbf{o}_{t+1} \sim Z(s_{t+1}, \mathbf{a}_t, \cdot)$ and global *shared* reward $r_t = R(s_t, \mathbf{a}_t, s_{t+1})$, as illustrated in Fig. 1. The objective is to learn a joint policy π^* that maximizes the expected discounted return $R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$.

A common way to address POMGs is through model-free reinforcement learning, where agents learn action-value functions directly from experience. Deep Q-Networks (DQN), one of the earliest pioneers of this approach uses a deep convolutional network to approximate the optimal action-value function $Q(s, a)$. The network parameters θ are optimized iteratively through the minimization of a differentiable loss function, calculated on mini-batches of state transitions (s, a, r, s') uniformly sampled $U(D)$ from a replay buffer D, yielding the temporal-difference loss

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(D)} \left[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2 \right]$$

Mnih et al. (2015). However, DQN assumes full observability and cannot recover hidden state information in partially observable settings such as plume tracing.

To overcome this, Hausknecht and Stone introduced the Deep Recurrent Q-Network (DRQN), which incorporates an LSTM that consumes each agent's observation history $H_t^i = \{o_1^i, \ldots, o_t^i\}$ and produces a hidden state h_t^i. The network then outputs Q-values $Q(o_t, h_t^i; \theta)$, which quantify the expected return of each action and thus identify the best action given the current observation and its history. As before, training occurs via the temporal-difference loss

$$L_{\text{DRQN}}(\theta) = \mathbb{E}_{(o,a,r,o') \sim U(D)} \left[(r + \gamma \max_{a'} Q(o', h'; \theta^-) - Q(o, h; \theta))^2 \right]$$

Hausknecht and Stone (2017). While this memory-augmented architecture enables agents to infer latent state dynamics from sequences of observations, it is well known that its single-network structure still suffers from overestimation bias as discussed in van Hasselt et al. (2015).

In this work, VanHasselt et al. extended deep Q-learning to the recurrent case with the Double Deep Recurrent Q-Network (DDRQN), which decouples action selection and evaluation: the online network picks $\arg\max_{a'} Q(o', h'; \theta)$, while a separate target network θ^- evaluates that action's value. This is seen via the new target value in the loss function:

$$y = r + \gamma\, Q\left(o', h'; \arg\max_{a'} Q(o', h'; \theta); \theta^-\right) \tag{1}$$

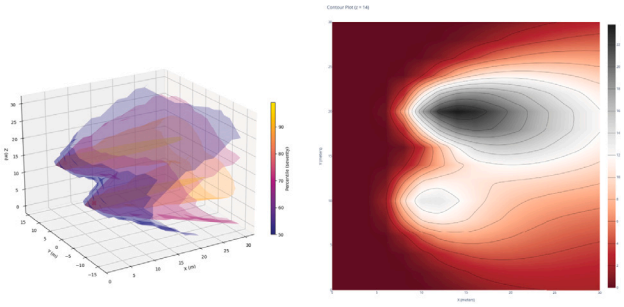
and training loss becomes:

$$L_{\text{DDRQN}}(\theta) = \mathbb{E}\left[(y - Q(o, h; \theta))^2 \right] \tag{2}$$

van Hasselt et al. (2015). This separation reduces overestimation and stabilizes learning under partial observability. We adopt both DRQN and DDRQN as baselines to compare it against our proposed ADDRQN. These baselines are trained by minimizing their respective losses on replay-buffer mini-batches as discussed above.

4. Gaussian plume dispersion model

Atmospheric dispersion refers to the diffusion of nonreactive pollutants under the influence of turbulent eddy motion and advection caused by meteorological conditions. In particular, Gaussian models (i.e. AERMOD, OCD, GPM, CTDMPLUS, etc.) have been widely adopted



(a) Volumetric isosurfaces of plume concentration with overlapping shells from distinct sources.

(b) 2D concentration contours at $z = 14$ m showing lateral plume overlap.

Fig. 2. Plume dispersion simulations using Eq. (4) with two point sources C_1 and C_2, each defined by distinct release parameters (u, Q, H) and stability classes (A/B). Overlap between plumes is evident in both the volumetric rendering (a) and the horizontal slice (b) which causes to **observation aliasing**: regions where identical concentration readings can result from different latent source configurations.

by EPA and other industrial operations as a standard approach for simulating the short-range transport of airborne pollutants. Gaussian Plume-based solutions have been extensively studied in a wide range of applications including multi-agent gas source detection (Wang & Wu, 2020), modeling realistic pollution in dense roadway interchanges (Middleton et al., 1979), and identifying livestock facility locations (Chastain & Wolak, 2000).

Under steady-state conditions ($\partial C/\partial t = 0$), a continuous point source of strength Q at $(0, 0, H)$, a constant wind $\mathbf{u} = (u, 0, 0)$, and isotropic eddy diffusivity $K(x)$ reduce the time-dependent advection–diffusion equation to

$$u\frac{\partial C}{\partial x} = K\left(\frac{\partial^2 C}{\partial y^2} + \frac{\partial^2 C}{\partial z^2}\right) + Q\,\delta(x)\,\delta(y)\,\delta(z-H).$$

Complete derivations using Laplace transforms and Green's functions can be found in Stockie (2011) and Veigele and Head (1978). Thus, the resulting closed-form analytical solution for a pollution concentration $C(x, y, z)$ is described by:

$$C(x,y,z) = \frac{Q}{2\pi u\sigma_y\sigma_z}\exp\left[-\left(\frac{y^2}{2\sigma_y^2}\right)\right]\times \left[\exp\left[\frac{-(z-H)^2}{2\sigma_z^2}\right] + \exp\left[\frac{-(z+H)^2}{2\sigma_z^2}\right]\right] \tag{3}$$

where σ_y and σ_z represent the respective lateral and vertical spreads normal to wind direction u. Eq. (3) can be further extended to model the coalesced pollution plume from multiple constituent point sources. Let n denote the total number of point sources, where $\vec{p}_i = (X_i, Y_i, H_i)$ is the location of the ith pollutant source with Q_i emission rate. Then the contaminant concentration at any point (x, y, z) is simply the superposition of their individual sources (Stockie, 2011)

$$C_T(x,y,z) = \sum_{s=1}^{n} C(x_i', y_i', z; Q_i, H_i) \tag{4}$$

where x_i' and y_i' are the shifted coordinates such that $x_i' = y_i' = 0$ for each i.

$$x_i' = x - X_i \text{ and } y_i' = y - Y_i$$

To train our multi-agent reinforcement models, we leveraged Eq. (4) to create multiple instances of pollution maps with varying meteorological conditions, number of point sources and emission rates. Fig. 2 displays a sample plume environment with two pollution sources along with its corresponding contour plot at a fixed height.

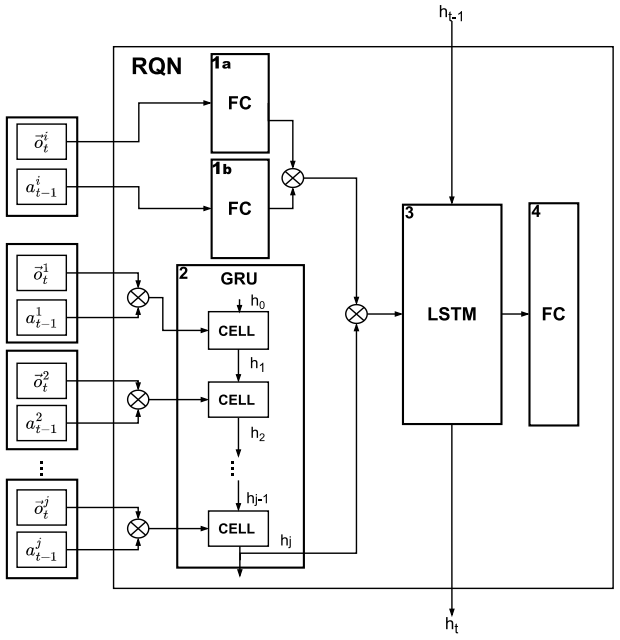


Fig. 3. Schematic of the action-specific multi-agent architectural backbone: (1a) Host-State Embedder and (1b) Host-Action Embedder embedding UAV observations and prior actions; (2) Other-Agent Embedder processing neighbor observation–action sequences; (3) Temporal LSTM Aggregator integrating fused embeddings over time; and (4) final Q-value output layer.

Table 1
Hyperparameters for MARL algorithm.

Hyperparameter	Value
Pollution noise severity (α)	.05
Learning rate (η)	0.0003
Soft-update rate (τ)	0.01
Batch size (B)	768
Sequence length (L)	75
Hidden size (H)	1024
Max epochs ($E_{\max}$)	2500
Discount factor (γ)	0.999
Epsilon start ($\epsilon_{\max}$)	1.0
Epsilon end ($\epsilon_{\min}$)	0.05
Decay rate (λ)	100
Exploration episodes (T_{init})	22 000
Total episodes (E)	23 000

5. ADDRQN approach

Reinforcement learning seeks to discover a policy π_θ that maps each agent's local information to a distribution over actions. Formally, for agent i at time t with observation s_t^i, a standard model computes $\pi(a_t^i \mid s_t^i)$. This ignores both the agent's own past actions and any information coming from teammates. Deep Recurrent Q-Networks (DRQN) address partial observability by embedding an agent's entire observation history, while the Action-Enhanced DRQN (ADRQN) of Zhu et al. couples single-agent observations and last actions into a unified LSTM input to improve latent-state inference in POMDPs. However, ADRQN remains strictly single-agent and assumes a stationary environment: its policy never sees or adapts to any other learning agents.

In contrast, we propose ADDRQN: a host-invariant, multi-agent action-specific Double Deep Recurrent Q-Network with a shared policy across all UAVs and designed to handle UAV faults such as intermittent communication loss, dynamic agent drop-in/drop-out, and noisy sensor readings. For any UAV i at time t, the policy takes the form

$$\pi^i\left(a_t^i \mid s_t^i,\ a_{t-1}^i,\ \{s_t^j, a_{t-1}^j\}_{j\neq i}\right),$$

where $s_t^i = (x_t^i, y_t^i, z_t^i, p(x_t^i, y_t^i, z_t^i))$ is the host agent's current state observation, a_{t-1}^i its previous action, and $\{s_t^j, a_{t-1}^j\}_{j\neq i}$ the state–action pairs of the other $N-1$ agents that is not the host ($i \neq j$).

5.1. Network architecture

At the heart of our method lies a shared-weight, host-invariant network backbone shown in Fig. 3. This architecture comprises of four constituents: (1a) a Host Observation Embedder and (1b) a Host Action Embedder, (2) an Neighboring-Agent GRU Embedder, (3) a Temporal Aggregator, and (4) an Action Output Head. The Host Observation Embedder and Host Action Embedder are small MLPs that respectively map UAV i's current state s_t^i and its previous action a_{t-1}^i into feature vectors. These are concatenated to form the host's state–action embedding. "Host" here does not imply a fixed identity; rather, any UAV can serve as host because the network parameters are shared and the architecture treats the host index as a permutation among agents. Concurrently, the Neighboring-Agent GRU Embedder concatenates each teammate j's state s_t^j and last action a_{t-1}^j into feature vectors, and encodes the unordered set

$$\mathcal{N}_t^i = \{(s_t^j, a_{t-1}^j)\}_{j\neq i}.$$

While GRUs are commonly used for temporal sequences, here we apply the GRU over the agent dimension, where $\mathcal{N}_t^i$ serves as the input sequence. This enables the module to process any number of teammates and produce a fixed-size embedding without padding or truncation.

The host and neighbor embeddings are concatenated and passed through a Temporal Aggregator (e.g. LSTM, GRU, or Transformer) applied over the fused feature sequence. Its role is to capture how the host and teammate information co-evolve over time. Finally, the Action Output Head, a small MLP, maps the aggregated representation to Q-values $Q^i(a)$, from which the selected action is given by $\arg\max_a Q^i(a)$. By construction, this architecture is host-invariant with respect to which UAV is treated as host and supports arbitrary team sizes at each timestep.

5.2. Training regime

Training unfolds over E episodes, each lasting up to $T_{\max}$ steps. At the start of episode e, we randomly sample a team size

$$k \sim \text{Uniform}\{1, \dots, N\}$$

and deploy k UAVs on a Gaussian-plume simulated environment with randomized initial positions (Alg. 1, lines 5–6).

We adopt an ϵ-greedy exploration strategy. During the first T_{init} episodes, $\epsilon = 1$, and as such all actions chosen are drawn uniformly at random. After an initial set of exploration episodes T_{init}, ϵ decays toward $\epsilon_{\min}$ according to

$$\epsilon(t) = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \exp\left(-\frac{t - T_{\text{init}}}{\lambda}\right) \tag{5}$$

gradually shifting the policy from pure exploration to near-greedy exploitation (line 17).

At each time step t, every UAV i receives a noisy observation

$$s_t^i = \left[x_t^i, y_t^i, z_t^i, p(x_t^i, y_t^i, z_t^i)\right] + \epsilon_t^i,$$

where $\epsilon_t^i \sim \mathcal{N}\left(0, (\alpha\, p(x_t^i, y_t^i, z_t^i))^2\right)$ simulates sensor noise proportional to the true concentration. The ADDRQN network then computes Q-values over the discrete action set {North, West, East, South, Up, Down, No Movement}, and each agent selects a_t^i via the current ϵ-greedy policy. Executing the joint action $\mathbf{a}_t$ yields next observations $\mathbf{s}_{t+1}$, a shared sparse reward

$$r_t = \begin{cases} 10 & \text{if a new plume is localized,} \\ 0 & \text{if the same plume is revisited,} \\ -1 & \text{otherwise,} \end{cases}$$

Algorithm 1: Multi-Agent Reinforcement Learning Pipeline with ADDRQNs

1 Initialize replay buffer buckets D_k for $k = 1, \dots, N$
2 Initialize online and target networks θ, θ^-
3 Initialize constants $T_{\text{init}}, \gamma, \tau, \alpha \dots$ (See Table 1)
4 **for** *episode* $e = 1$ **to** E **do**
5 Randomly choose number of UAVs $k \in \{1, \dots, N\}$ for this episode
6 Randomly select pollution map and initial positions for the k UAVs. This sets our initial observation $\mathbf{o}_t$
7 Set hidden state $h^i \leftarrow 0$ and last action $a_{t-1}^i \leftarrow 0$
8 Inject Gaussian noise $\nu_t \sim \mathcal{N}(0, (\alpha\,|C_T(p_t)|)^2)$ into entire pollutant field
9 **for** *step* $t = 1$ **to** $T_{\max}$ **do**
10 For each agent $i = 1, \dots, k$: select action a_t^i using the ϵ-greedy policy
11 Execute joint action $\mathbf{a}_t = (a_t^1, \dots, a_t^k)$
12 Observe next observations $\{o_{t+1}^i\}$, rewards $\{r_t^i\}$, and done flags $\{\text{done}^i\}$
13 Append $(\mathbf{a}_{t-1}, \mathbf{o}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{o}_{t+1}, \text{done})$ to bucket k in replay buffer D_k
14 Update observation $\mathbf{o}_t \leftarrow \mathbf{o}_{t+1}$
15 Update prior action $\mathbf{a}_{t-1} \leftarrow \mathbf{a}_t$ for each agent i
16 **if** *episode* $> T_{init}$ **then**
17 Update $\epsilon \leftarrow \epsilon_{\text{new}}$ with ϵ-decay using Eq. (5)
18 Randomly sample team size $k \in \{1, \dots, N\}$ and select bucket D_k
19 Sample minibatch of B sequences (length L) from D_k
20 Sample host index $i \in \{1, \dots, k\}$ for the entire minibatch
21 Partition all B sequences into host pairs (s, a) and neighbor sets $\{(s, a)\}_{j\neq i}$
22 Compute target y via Eq. (1) and loss L via Eq. (2)
23 Update online network parameters using back propagation and optimizer
24 Update target network parameters using soft update $\theta^- = \tau\,\theta + (1-\tau)\,\theta^-$
25 **if** *any done*^{i} **then**
26 **break**

and episode termination flags (lines 10–12).

The full team experience at time t is recorded as a composite transition (line 13):

$$\left(\mathbf{a}_{t-1}, \mathbf{S}_t, \mathbf{a}_t, r_t, \mathbf{S}_{t+1}, \mathbf{d}_t\right),$$

where $\mathbf{a}_{t-1}, \mathbf{a}_t \in \{1, \dots, |\mathcal{A}|\}^k$ are the prior and current action vectors, $\mathbf{S}_t, \mathbf{S}_{t+1} \in \mathbb{R}^{k\times 4}$ are state matrices, r_t is the shared reward scalar, and $\mathbf{d}_t \in \{0, 1\}^k$ contains the agent-specific termination flags. Since the dimensionality of each transition depends on team size k, we maintain separate replay buffers $D_1, \dots, D_N$, one for each possible value of k.

Once the exploration phase ends ($e > T_{\text{init}}$), every environment interaction triggers a learning update (lines 16–24). At each update step, we *uniformly* sample one of the N replay buckets D_k (resampling team size) and then draw B sequences of length L from that bucket (lines 18–19). To construct host-invariant inputs, we uniformly select a single host index $i \in \{1, \dots, k\}$ for the minibatch and partition each tuple into the host pair (s_t^i, a_{t-1}^i) and the neighbor set $\{(s_t^j, a_{t-1}^j)\}_{j\neq i}$ (lines 20–21).

These host and neighbor representations are passed through the ADDRQN network to compute Q-values, which are used to construct training targets via Eq. (1) (line 22). We compute loss according to

Table 2
Ablation-study architectures.

Phase	IDs	Host Emb	Neighbor Emb	Temporal	Output
1	1–4	MLP-32 × 1	LSTM-64 × 1	LSTM-512 × 1	MLP-7 × 1
	5–8	MLP-64 × 1	LSTM-128 × 1	LSTM-1024 × 1	MLP-7 × 1
	9–12	MLP-128 × 1	LSTM-256 × 1	LSTM-2048 × 1	MLP-7 × 1
2	13–16	MLP-32 × 1	LSTM-128 × 1	LSTM-1024 × 1	MLP-7 × 1
	17–20	MLP-64 × 1	LSTM-64 × 1	LSTM-1024 × 1	MLP-7 × 1
	21–24	MLP-64 × 1	LSTM-128 × 1	LSTM-512 × 1	MLP-7 × 1
3	25–28	MLP-64 × 1	GRU-128 × 1	LSTM-1024 × 1	MLP-7 × 1
	29–32	MLP-64 × 1	LSTM-128 × 1	GRU-1024 × 1	MLP-7 × 1

Notation. "MLP-$d\times\ell$" = ℓ fully connected layer(s) of width d. "LSTM-$d\times\ell$"/"GRU-$d\times\ell$" = recurrent layer(s) with d hidden units. The output head is an MLP (e.g., MLP-7 × 1); the variant "MLP-64 × 1→7 × 1" adds a 64-unit pre-head before the 7-way output. ID ranges (e.g., 5–8) expand to four RL types in fixed order: *ADDRQN*, *ADRQN*, *DDRQN*, *DRQN* (with +0, +1, +2, +3 ID offsets). **Example.** "5–8" corresponds to a host embedder that is a fully connected network with one layer of width 64; an "other" embedder that is an LSTM with 128 hidden units and one layer; a temporal module that is an LSTM with 1024 hidden units and one layer; and an output head that is a single-layer MLP with 7 units.

Eq. (2), backpropagate gradients through the online network, and update the target network via a soft update:

$$\theta^- \leftarrow \tau\,\theta + (1-\tau)\,\theta^-,$$

where $\tau \ll 1$ determines the smoothing factor (lines 23–24).

This training procedure serves two core objectives. First, by sampling both the team size k and the host index i at each update, a single Q-network is trained to accommodate variable team sizes and interchangeable host roles. This enforces host-invariance and allows the model to generalize across agent configurations. Second, injecting zero-mean Gaussian noise during training regularizes the value function against sensor drift and transient spikes. As a result, these mechanisms enable ADDRQN to robustly handle UAV failure, intermittent blackout communications, and noisy observations as corroborated by our Evaluation section.

6. Evaluation of MARL framework

We evaluate our MARL framework along three dimensions: the architectural impact of action-specific embeddings, policy robustness under realistic failure scenarios, and the effect of team size on performance. Unless stated otherwise, performance is assessed using two primary metrics: **Success Rate**, defined as the percentage of trials in which all pollution sources are localized within a 600-step budget; and **Episode Length**, defined as the average number of steps required to complete successful missions. All trials are drawn from a held-out test set with randomized initial states and sensor noise. Training and evaluation use the hyperparameters listed in Table 1. The following research questions structure our analysis:

1. **RQ1:** How does including each agent's previous action a^i_{t-1} in ADRQN and ADDRQN affect mission success and efficiency relative to their non-action counterparts?
2. **RQ2:** How robust is our MARL framework in the presence of UAVs mid-mission drop out or temporary and intermittent communication blackouts?
3. **RQ3:** How does team size $k \in \{1,2,3,4\}$ influence success rate and time-to-localize? Our aim is to discern whether multi-agents settings improve success and/or efficiency in plume tracing tasks.

6.1. RQ1: Ablation study

Our architecture comprises four modules: host-agent embedders (includes both observation and action layers), neighboring-agent embedder, temporal aggregator, and the Q-value output head (Fig. 3). In this section, we investigate the performance impact of action-specific models by evaluating multiple architectural configurations. Each architecture is tested with four model types: ADDRQN, ADRQN, DRQN, and DDRQN, which yields 32 unique experiments (Table 2). We structure the ablation into three phases:

1. **Global Width Ablation.** This phase evaluates whether action-specific benefits persist across model scales. All modules share a uniform width per setting, tested at three tiers: low ([32, 64, 512]), medium ([64, 128, 1024]), and high ([128, 256, 2048]).
2. **Per-Component Width Ablation.** To identify architectural bottlenecks, we halve the width of a single module while keeping all others fixed at default (mid-size tier). This isolates the contribution of each module to overall performance and reveals sensitivity to size.
3. **Module Replacement Ablation.** Here, we replace the temporal aggregator and neighbor-agent embedder with GRU modules. This isolates architectural gains that result from design differences rather than from the use of action inputs.

Across all phases, we evaluate each architecture using 5000 test trials drawn from a held-out evaluation set composed of environments unseen during training. As in training, each pollution reading is perturbed by Gaussian noise proportional to its true value to simulate sensor uncertainty. For each trial, team size is randomly sampled from {1, 2, 3}, and the starting positions of agents are selected at random. All test-time variables (pollution field, noise realizations, team composition, and initial locations) are held fixed across models using a shared random seed to ensure fair comparison. We report success rate, defined as the fraction of trials where all sources are detected within a step budget of 600 as the primary evaluation metric.

The results are summarized in Fig. 4. Across the 32 experiments, *action-specific* + *double Q* (ADDRQN) emerges as the most reliable configuration and achieves the highest success rate across all architectural variants (ID 25 at 94.3%). In Phase 1, we observe a clear performance optimum at medium capacity. Moving from the low to medium tier significantly improves ADDRQN performance (64.2% to 83.0%), while scaling further to the high tier causes a drop (69.9%). ADRQN exhibits a similar but sharper decline, peaking at 76.0% before collapsing at higher widths. DRQN and DDRQN follow the same trajectory, with large models failing to retain earlier gains. These outcomes reflect a classic bias–variance trade-off where medium-width networks offer sufficient capacity to leverage action inputs while avoiding the instability that arises from excessive parameterization. Notably, ADDRQN maintains higher performance than ADRQN even at larger scales. This suggests that double Q-learning plays a stabilizing role by tempering overestimation and limiting oscillations that become more pronounced with excess capacity.

The per-component width ablations (phase 2) reveal where capacity matters most. Reducing either the host-agent embedder or the agent aggregator has little consequence for ADDRQN which suggests both modules have sufficient slack capacity under the default configuration. In contrast, halving the temporal aggregator is the most consequential change. ADDRQN falls to 71.2% and the margin to the non–action-aware baselines narrows, which indicates that the recurrent module is the primary conduit through which action history becomes useful for prediction. DDRQN remains volatile across these configurations, which further suggests that without explicit action inputs the model cannot reliably stabilize learning.

In Phase 3, we swap the LSTM components for GRUs to assess the effect of module type independent of width. While both GRU substitutions outperform their LSTM counterparts, the larger effect arises when the GRU is used for the neighbor-agent embedder (reaches 94.3%, ID 25), whereas replacing the temporal module yields a smaller but still positive gain (peaks at 89.4%, ID 29). Two factors may help explain this difference. At the agent level, Teammate observations can be noisy

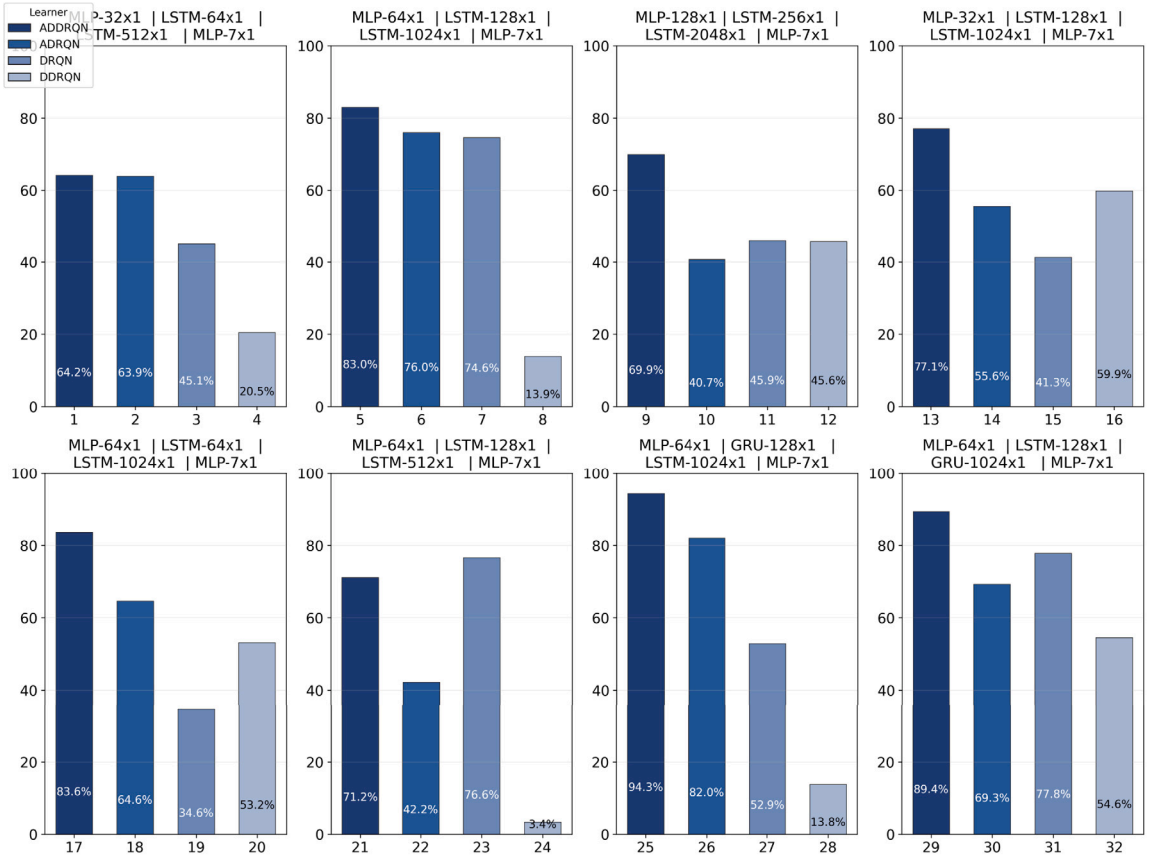
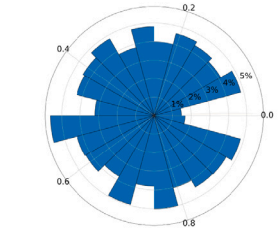


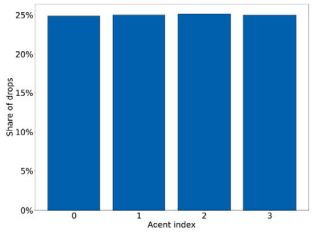
Fig. 4. Success rate per architecture grouped by ablation phase (IDs 1–4, 5–8, etc.), with bars color-coded by model type. Results demonstrate that action-based models consistently outperform their observation-only counterparts across all configurations. In particular, ADDRQN dominates, ranking first in 7 of 8 groups and peaking at 94.3% success, while DDRQN consistently underperforms. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

and inconsistent due to augmented sensor noise, randomized team composition, and host randomization during training. GRU gating acts as a selective filter that emphasizes consistent signals while suppressing noise. At the temporal level, the GRU's lighter gating and reduced parameterization ease optimization under bootstrapped targets and enhance stability, though with weaker long-horizon memory than an LSTM (hence modest benefit). The baseline models follow a similar trend, where ADRQN displays its highest performance with 82% (ID 26), and DRQN peaks at 74.3% (ID 31). While DDRQN remains unstable across both replacements which suggests architectural swaps do not offset its inherent volatility.

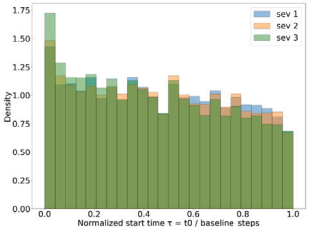
Looking across all phases, several consistent patterns emerge. DDRQN fails to stabilize across any settings and never exceeds a 60% success rate. In contrast, ADDRQN is consistently the most reliable learner, which attains the highest score in 7 of 8 configurations and peaking at 94.3% (ID 25). The benefit of incorporating action history follows from reduced perceptual aliasing. In plume tracing tasks, the same observation may correspond to different latent states depending on the agent's previous action. Conditioning the embedding on that prior action disambiguates these cases and yields representations in which the Q-function is easier to approximate and temporal credit assignment becomes more accurate. Although less pronounced, this effect is also evident in ADRQN, which outperforms its action-agnostic baseline in 5 out of 8 configurations. This further indicates that Double-Q further stabilizes training by mitigating overestimation and curbing destabilizing policy oscillations.



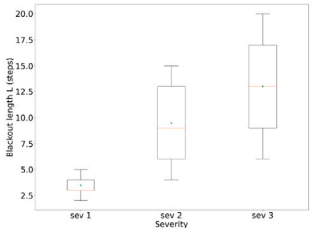
(a) **Dropout: normalized drop time** ($\tau = t_{\text{drop}}/T_0$). Circular histogram shows a near-uniform distribution of dropout events over the episode timeline.



(b) **Dropout: dropped agent index distribution**. Histogram of selected dropout indices across agents. Uniformity confirms agent-agnostic removal with no role-dependent bias.



(c) **Blackout: start time** ($\tau = t_0/T_0$). Distribution of blackout onset times. The mild bias toward early starts is offset by broad coverage.



(d) **Blackout: duration L by severity** s. Boxplots confirm increasing median and variance in blackout lengths across severity levels $s \in \{1, 2, 3\}$.

Fig. 5. RQ2: Distribution checks for dropout and blackout injection. Diagnostic visualizations confirm unbiased failure injection across time, agent identity, and severity levels.

Table 3
RQ2 robustness summary by scenario.

Scenario	ℓ	n	SR/B	ΔSR (%)	Steps (Md [Q1,Q3])
Dropout	1	15k	0.988	1.20	1.000 [1.000, 1.368]
	2	15k	0.972	2.80	1.286 [1.250, 2.069]
	3	15k	0.869	13.1	2.269 [2.071, 5.464]
Blackout	1	15k	0.9982	0.17	1.000 [1.000, 1.000]
	2	15k	0.9976	0.23	1.000 [0.933, 1.000]
	3	15k	0.9984	0.15	1.000 [0.838, 1.077]

Notation: n = number of evaluation episodes; ℓ = severity level index (**Dropout**: number of agents removed; **Blackout**: severity level); **SR/B** = success rate relative to baseline; ΔSR (%) = $100\,(1 - \text{SR/B})$; **Steps**×= episode-length ratio vs. baseline (Md = median, Q1/Q3 = quartiles).

6.2. RQ2: Robustness to UAV failures and communication loss

Small UAVs, while highly capable, are also prone to brittleness in real-world deployments. These multidisciplinary platforms face a wide range of hardware faults, including propulsion or ESC glitches, motor brownouts, IMU and magnetometer drift, GPS dropout, barometric sensor anomalies, and thermal throttling of onboard compute. In severe cases, such failures render the drone inoperable, forcing it to abort its mission. Compounding this challenge, UAVs often operate in unpredictable environments with unreliable wireless networks subject to bursty packet loss, RF interference, and range-limited communication.

If such contingencies are not accounted for during policy design, system-level performance and inter-agent coordination can degrade significantly. In multi-agent reinforcement learning (MARL), where agents leverage teammate signals for cooperative behavior, missing or corrupted inputs can lead to brittle execution or even policy collapse. To address this, centralized training with decentralized execution (CTDE) is commonly employed. In CTDE, agents learn with access to privileged global information, but at execution time, policies must operate using only local observations and limited peer visibility. This regime prepares agents to remain functional even when neighboring units fail or become temporarily unreachable.

To evaluate the robustness of our trained ADDRQN policy under such failure modes, we simulate two types of disruption: (i) permanent UAV dropouts due to critical hardware failure, and (ii) transient loss of inter-agent communication due to networking blackouts. Both are implemented by manipulating the per-agent visibility set $\mathcal{V}_t^i$, which governs the set of peer observations available at timestep t.

Let there be N agents, indexed by $i \in \{1, \dots, N\}$, each running an identical instance of the trained shared-weight policy π_θ. At timestep t, agent i observes $o_t^i \in \mathbb{R}^{d_o}$ and remembers its previous action $a_{t-1}^i \in \{1, \dots, A\}$. The policy outputs the next action as

$$a_t^i \sim \pi_\theta\Big(o_t^i,\ a_{t-1}^i,\ \{(o_t^j, a_{t-1}^j)\}_{j \in \mathcal{V}_t^i}\Big),$$

where $\mathcal{V}_t^i \subseteq \{1, \dots, N\} \setminus \{i\}$ denotes the set of teammates visible to agent i at time t. The construction of $\mathcal{V}_t^i$ differs depending on the type of fault being emulated and allows us to induce both dropout and communication failure scenarios in a controlled and reproducible manner.

6.2.1. Intermittent communication loss

To emulate unreliable peer-to-peer communication links, we introduce intermittent blackouts by dynamically pruning the visibility set of affected agents during runtime. At each timestep t, every agent independently has a probability $p_{\text{start}}^{(s)}$ of initiating a blackout, where severity level $s \in \{1, 2, 3\}$ controls this rate. Once triggered, the blackout duration is sampled uniformly from a predefined severity-specific range, $L^{(s)} \in [\underline{L}^{(s)}, \overline{L}^{(s)}]$. Let $\mathcal{B}_t$ denote the set of agents experiencing a blackout at time t. We define the visibility set $\mathcal{V}_t^i$ of host agent i as follows:

$$\mathcal{V}_t^i = \begin{cases} \varnothing, & \text{if } i \in \mathcal{B}_t \quad \text{(host in blackout sees no peers)}, \\ \{\, j \neq i \mid j \notin \mathcal{B}_t \,\}, & \text{if } i \notin \mathcal{B}_t \quad \text{(exclude blacked-out neighbors)}. \end{cases}$$

Under this scheme, agents undergoing a blackout make decisions based solely on their own observation–action pair (o_t^i, a_{t-1}^i) without neighbor context, while unaffected agents retain partial coordination by observing only peers that are currently online. As severity s increases, both the likelihood and duration of blackouts grow, which leads to more severe and prolonged disruptions.

To quantify policy resilience under these conditions, we compare performance against a non-perturbed baseline in which no blackouts occur, i.e., $\mathcal{B}_t = \varnothing$ for all t, and full inter-agent visibility is preserved throughout the episode. This serves as the idealized case with uninterrupted communication. As shown in Table 3, success rates remain above 99.75% for all severity levels, corresponding to less than 0.25% degradation relative to the baseline. Likewise, median episode durations remain flat at 1.0×, indicating no measurable slowdown despite prolonged losses in inter-agent observability. The histogram of blackout initiation times (Fig. 5(c)) reveals a mild bias toward earlier timesteps, though blackout onsets span the full episode length. These results confirm that the observed robustness is genuine and consistent across time, with performance unaffected by when or how frequently communication blackouts arise.

6.2.2. Severe drone hardware failure

To evaluate policy robustness under catastrophic hardware failure, we simulate permanent agent dropouts where one or more UAVs become inoperable and withdraw from the mission. Such failures may result from propulsion loss, onboard compute throttling, or sensor collapse. We model this by sampling a discrete dropout time $t_{\text{drop}} \sim \text{Unif}\{0, 1, \dots, T_0 - 1\}$ at the start of each episode, where T_0 is the nominal episode length. At this time, a victim agent j is drawn uniformly at random from the active set. For all timesteps $t \geq t_{\text{drop}}$, agent j is removed from the team, which we enforce by updating the visibility sets of all remaining agents such that:

$$\mathcal{V}_t^i \subseteq \{1, \dots, N\} \setminus \{i, j\} \qquad \text{and} \qquad j \notin \mathcal{V}_t^i \quad \forall\, i \neq j.$$

From that point onward, each active agent i continues to act based on its own observation–action history (o_t^i, a_{t-1}^i), while aggregating context only over the reduced visibility set $\mathcal{V}_t^i$. That is, all coordination occurs with one fewer peer than before the failure. We evaluate three severity levels $s \in \{1, 2, 3\}$, where s denotes the number of agents guaranteed to be dropped per episode. In the most extreme case, $s = 3$, only one agent remains to complete the mission.

To quantify the effect of team reduction mid-mission, we compare results against a non-dropout baseline in which all N agents remain functional and visible throughout the episode. This baseline represents the ideal case with perfect team integrity, no structural perturbations, and full inter-agent observability at all times.

As summarized in Table 3, the ADDRQN policy exhibits graceful degradation under increasing dropout severity. For $k = 1$ and $k = 2$, success rates decline modestly by 1.20% and 2.80%, respectively, and the median episode lengths rise to 1.00× and 1.29× the baseline. When three agents are dropped ($k = 3$), only one UAV remains to explore the environment, which causes success rate to fall to 86.9% and median step count to more than double (2.27×). This reflects the inherent difficulty of single-agent search, but also confirms that the policy remains functional and capable of localization under extreme team reduction. In addition, both the rose plot of normalized drop times (Fig. 5(a)) and the agent index histogram (Fig. 5(b)) display uniform distributions. This confirms the dropout injection mechanism is unbiased with respect to time and agent identity, and that policy resilience is not an artifact of structural asymmetries in agent roles. These findings validate the efficacy of host-randomized CTDE training in equipping all agents with role-invariant, fault-tolerant policies.

Table 4
Per-team-size evaluation metrics on the multi-agent benchmark.

k	n	SR (%)	$\overline{L}$	Md [Q1, Q3]
1	1490	88.93	147.33	57.00 [36.00, 160.00]
2	1473	95.25	75.26	36.00 [26.00, 53.00]
3	1553	96.72	54.16	28.00 [21.00, 40.00]
4	1484	97.30	43.77	23.50 [19.00, 32.00]

Notation: k = team size (agents); n = number of episodes; **SR** = success rate; $\overline{L}$ = mean episode length (steps); **Md** [**Q1**, **Q3**] = median and quartiles of episode length (steps).

6.3. RQ3: Per team size multiagent

In this section, we assess whether increasing the number of agents improves success rate and efficiency. We evaluate the trained ADDRQN policy on 6000 held-out test trials, with each pollution map augmented by sensor noise and randomized agent start positions. For each trial, the team size $k \in \{1, 2, 3, 4\}$ is sampled uniformly. We record whether all sources are found within a 600-step budget (success rate) and the number of steps taken (episode length). Results are grouped by team size and summarized in Table 4.

Results show a clear upward trend with team size: success increases from 88.93% at $k = 1$ to 97.30% at $k = 4$ with the largest improvement from $k = 1$ to $k = 2$ (+6.3 pp) and mean steps decrease from 147.33 to 43.77 with about half the steps at $k = 2$ and smaller reductions thereafter. These improvements are expected, more agents enable parallel exploration, increase spatial coverage which leads to better performance.

7. Limitations and future directions

One limitation of our approach is the reliance on a Gaussian Plume Model (GPM) to simulate gas dispersion. The GPM has indeed been a popular choice in both environmental engineering and robotics due to its simplicity and analytical tractability. The advantages of the Gaussian Dispersion Model (GDM) include its simplicity, computational efficiency, and effectiveness in short to medium-range dispersion (up to 20 km) over flat, homogeneous terrains with continuous emissions under neutral and stable atmospheric conditions. This simplicity has made the GPM effective in many real-world scenarios. For example, in indoor robotics, Sánchez-Sosa et al. (2018) demonstrated that a mobile robot using a GPM-based observation model could identify the location of a gas source with only 14 cm of error. In emergency response, Lotrecchiano et al. (2020) used a Gaussian plume approximation to back-estimate emissions from an industrial fire based on $PM_{2.5}$ measurements. In radiological safety, Cao et al. (2020) incorporated GPM into the core of the RADC simulation tool and validated its outputs against the HotSpot model in order to support rapid consequence assessment and probabilistic modeling.

These examples support the use of GPM as a first-order approximation in scenarios where assumptions are satisfied (i.e., steady emission, flat terrain, and homogeneous flow). However, these same assumptions limit the model's applicability in complex terrain, urban environments, and nonstationary meteorological regimes. For example, in mountainous or urban areas, the simplifying assumptions break down due to the presence of complex obstacles and variable winds. Complex terrain like valleys and hills can cause plume deflection, channeling, or trapping that a straight-line Gaussian model cannot capture. In particular, regulatory Gaussian-based models have been shown to misrepresent ground-level concentrations in complex topographies. For instance, the CALPUFF Gaussian puff model substantially under-predicted pollutant concentrations in a deep valley (Adige Valley, Italy) because it could not account for the terrain-induced flow patterns (Giovannini et al., 2020; Zardi et al., 2021). Consistent with these observations, Pirhalla et al. show in a mock irregular urban array that a steady-state Gaussian model (AERMOD) qualitatively reproduces the ground-level plume but underestimates peak concentrations, overestimates lateral spread, and struggles in the very near field (Pirhalla et al., 2021). Even after adding an LES-derived turbulence profile, far-field peaks remain underpredicted, and under oblique winds most Gaussian models are likely to fail due to channeling and off-axis plume drift. As a result, our trained policy may underperform in environments with complex terrain, urban canyons, or nonstationary winds. In such situations, more advanced dispersion models (e.g. puff models, Lagrangian particle models or CFD-based simulations) are recommended to improve accuracy.

Another limitation of our current approach is the absence of an explicit collision-avoidance term in the training reward function or policy. In the present formulation, the agent is not directly penalized for collisions with obstacles and rather focuses solely on plume tracking and source seeking. This poses safety concerns, given that a policy that greedily maximizes information gain or speed could inadvertently collide with obstacles if not trained to avoid them. In practical robotic deployments, navigation is inherently a multi-objective problem, that is, the robot must seek the target (gas source) while also ensuring safety. One relatively straight-forward direction is to incorporate obstacle avoidance as a secondary objective in the reinforcement learning framework. For instance, researchers have applied multi-objective reinforcement learning to train agents that balance goal-seeking with collision avoidance by optimizing a combined reward function (Ramezani Dooraki & Lee, 2022). An alternative (or complementary) strategy is to draw on concepts from safe reinforcement learning, which introduce explicit safety constraints or post-processing of actions. For example, Ahn et al. (2022) propose augmenting a standard RL agent with a safety filter that monitors the agent's chosen actions and overrides or reshapes any action predicted to cause a collision (Ahn et al., 2022). This type of safety layer functions as an external controller that intervenes on unsafe actions, but to ensure compatibility and optimal performance, it should be integrated during training so the policy can learn to operate under its constraints.

8. Conclusion

We address the problem of multi-source plume tracing in three-dimensional environments by modeling it as a cooperative partially observable Markov game (POMG). In this setting, agents must act under incomplete and noisy observations, with no direct access to global state or the true plume configuration. Observability is further complicated by the sporadic nature of gas encounters, which occur in brief, intermittent patches rather than continuous gradients. At the same time, agents operate in dynamic teams where both the number and identity of participants may vary due to hardware failures, communication disruptions, or mission design. These factors demand policies that are robust not only to uncertainty but also to changes in team composition and coordination patterns.

To this end, we propose ADDRQN: a host-invariant, action-conditioned recurrent value function trained under a randomized centralized training and decentralized execution (CTDE) regime. Our network processes each agent's current observation alongside its previous action to improve latent-state inference and disambiguate plume structure under partial observability. Teammate information is encoded through a permutation-invariant set module that accommodates varying numbers of collaborators without assuming fixed roles or team sizes. This design enables a single shared policy to generalize across variable deployments while remaining robust to drop- in/drop-out and host reassignment during execution.

Empirical evaluation confirms that conditioning on action-observation pairs improves inference and leads to higher success rates and shorter episodes compared to non-action baselines (RQ1). When communication links are interrupted or UAVs become inoperable, the policy remains robust and has a negligible performance degradation (RQ2). Finally, adding more agents provides consistent improvements,

with the largest gain observed when moving from one to two participants, followed by smaller but steady benefits as team size increases (RQ3). These results show that action-conditioned recurrent policies trained with randomized CTDE achieve both adaptability and robustness under uncertainty.

CRediT authorship contribution statement

Pedro Antonio Alarcon Granadeno: Conceptualization, Methodology, Software, Investigation, Formal analysis, Writing – original draft, Writing – review & editing. **Theodore Chambers:** Writing – review & editing. **Jane Cleland-Huang:** Funding acquisition, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work in this paper has been funded by the US National Science Foundation (NSF) under Grant #NSF-1931962.

Data availability

Data will be made available on request.

References

Ahn, H., Lim, C., Lee, J. D., & Bang, H. (2022). Safe reinforcement learning based multi-rotor collision avoidance with unexpected obstacles. In *33rd congress of the international council of the aeronautical sciences* (pp. 5663–5673). International Council of the Aeronautical Sciences, Publisher Copyright: © 2022 ICAS. All Rights Reserved.; 33rd Congress of the International Council of the Aeronautical Sciences, ICAS 2022 ; Conference date: 04-09-2022 Through 09-09-2022.

Alvear, O., Calafate, C. T., Zema, N. R., Natalizio, E., Hernández-Orallo, E., Cano, J.-C., & Manzoni, P. (2018). A discretized approach to air pollution monitoring using UAV-based sensing. *Mobile Networks and Applications*, *23*(6), 1693–1702. http://dx.doi.org/10.1007/s11036-018-1065-4.

Babaeizadeh, M., Frosio, I., Tyree, S., Clemons, J., & Kautz, J. (2017). Reinforcement learning through asynchronous advantage actor-critic on a GPU. arXiv:1611.06256. URL: https://arxiv.org/abs/1611.06256.

Cao, B., Cui, W., Chen, C., & Chen, Y. (2020). Development and uncertainty analysis of radionuclide atmospheric dispersion modeling codes based on Gaussian plume model. *Energy*, *194*, Article 116925. http://dx.doi.org/10.1016/j.energy.2020.116925, URL: https://www.sciencedirect.com/science/article/pii/S0360544220300323.

Chastain, J., & Wolak, F. (2000). Application of a Gaussian plume model of odor dispersion to select a site for livestock facilities. *Proceedings of the Water Environment Federation*, *2000*, 745–758. http://dx.doi.org/10.2175/193864700785303321.

Chen, X., Fu, C., & Huang, J. (2021). A deep Q-network for robotic odor/gas source localization: Modeling, measurement and comparative study. *Measurement*, *183*, Article 109725. http://dx.doi.org/10.1016/j.measurement.2021.109725.

Clark, J., & Fierro, R. (2005). Cooperative hybrid control of robotic sensors for perimeter detection and tracking. In *Proceedings of the 2005, American control conference, 2005* (pp. 3500–3505). http://dx.doi.org/10.1109/ACC.2005.1470515, vol. 5.

Francis, A., Li, S., Griffiths, C., & Sienz, J. (2022). Gas source localization and mapping with mobile robots: A review. *Journal of Field Robotics*, *39*(8), 1341–1373. http://dx.doi.org/10.1002/rob.22109, arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/rob.22109. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/rob.22109.

Gielis, J., Shankar, A., & Prorok, A. (2022). A critical review of communications in multi-robot systems. *Current Robotics Reports*, *3*(4), 213–225. http://dx.doi.org/10.1007/s43154-022-00090-9, URL: https://link.springer.com/article/10.1007/s43154-022-00090-9.

Giovannini, L., Ferrero, E., Karl, T., Rotach, M. W., Staquet, C., Trini Castelli, S., & Zardi, D. (2020). Atmospheric pollutant dispersion over complex terrain: Challenges and needs for improving air quality measurements and modeling. *Atmosphere*, *11*(6), http://dx.doi.org/10.3390/atmos11060646, URL: https://www.mdpi.com/2073-4433/11/6/646.

Hausknecht, M., & Stone, P. (2017). Deep recurrent Q-learning for partially observable MDPs. arXiv:1507.06527. URL: https://arxiv.org/abs/1507.06527.

Hayes, A., Martinoli, A., & Goodman, R. (2002). Distributed odor source localization. *IEEE Sensors Journal*, *2*(3), 260–271. http://dx.doi.org/10.1109/JSEN.2002.800682.

Hu, H., Song, S., & Chen, C. L. P. (2019). Plume tracing via model-free reinforcement learning method. *IEEE Transactions on Neural Networks and Learning Systems*, *30*(8), 2515–2527. http://dx.doi.org/10.1109/TNNLS.2018.2885374.

Li, W., Farrell, J., Pang, S., & Arrieta, R. (2006). Moth-inspired chemical plume tracing on an autonomous underwater vehicle. *IEEE Transactions on Robotics*, *22*(2), 292–307. http://dx.doi.org/10.1109/TRO.2006.870627.

Loisy, A., & Heinonen, R. A. (2023). Deep reinforcement learning for the olfactory search POMDP: a quantitative benchmark. *The European Physical Journal E*, *46*(3), http://dx.doi.org/10.1140/epje/s10189-023-00277-8.

Lotrecchiano, N., Sofia, D., Giuliano, A., Barletta, D., Poletto, M., et al. (2020). Pollution dispersion from a fire using a Gaussian plume model. *International Journal of Safety and Security Engineering*, *10*, 431–439.

Mayhew, C. G., Sanfelice, R. G., & Teel, A. R. (2007). Robust source seeking hybrid controllers for autonomous vehicles. In *NULL, Proc. 26th American control conference* (pp. 1185–1190). doi:http://www.scivee.tv/node/2725,http\protect\protect\leavevmode@ifvmode\kern+.2222em\relax//ieeexplore.ieee.org/iel5/4282134/4282135/04283016.pdf?tp=&isnumber=4282135&arnumber=4283016&punumber=%3Cb%3E%3Cfont%20color=990000%3E4282134%3C/font%3E%3C/b%3E. URL: https://hybrid.soe.ucsc.edu/files/preprints/12.pdf.

Middleton, D., Butler, J., & Colwill, D. (1979). Gaussian plume dispersion model applicable to a complex motorway interchange. *Atmospheric Environment (1967)*, *13*(7), 1039–1049. http://dx.doi.org/10.1016/0004-6981(79)90014-3, URL: https://www.sciencedirect.com/science/article/pii/0004698179900143.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, *518*(7540), 529–533. http://dx.doi.org/10.1038/nature14236.

Neumann, P. P., Bennetts, V. H., Lilienthal, A. J., Bartholmai, M., & Schiller, J. H. (2013). Gas source localization with a micro-drone using bio-inspired and particle filter-based algorithms. *Advanced Robotics*, *27*(9), 725–738. http://dx.doi.org/10.1080/01691864.2013.779052, arXiv:https://doi.org/10.1080/01691864.2013.779052.

Petritoli, E., Leccese, F., & Ciani, L. (2018). Reliability and maintenance analysis of unmanned aerial vehicles. *Sensors*, *18*(9), http://dx.doi.org/10.3390/s18093171, URL: https://www.mdpi.com/1424-8220/18/9/3171.

Pirhalla, M., Heist, D., Perry, S., Tang, W., & Brouwer, L. (2021). Simulations of dispersion through an irregular urban building array. *Atmospheric Environment*, *258*, Article 118500. http://dx.doi.org/10.1016/j.atmosenv.2021.118500.

Puterman, M. L. (1990). Chapter 8 Markov decision processes. In *Handbooks in operations research and management science*: *vol. 2*, *Stochastic models* (pp. 331–434). Elsevier, http://dx.doi.org/10.1016/S0927-0507(05)80172-0, URL: https://www.sciencedirect.com/science/article/pii/S0927050705801720.

Ramezani Dooraki, A., & Lee, D.-J. (2022). A multi-objective reinforcement learning based controller for autonomous navigation in challenging environments. *Machines*, *10*(7), http://dx.doi.org/10.3390/machines10070500, URL: https://www.mdpi.com/2075-1702/10/7/500.

Sánchez-Sosa, J. E., Castillo-Mixcóatl, J., Beltrán-Pérez, G., & Muñoz-Aguirre, S. (2018). An application of the Gaussian plume model to localization of an indoor gas source with a mobile robot. *Sensors*, *18*(12), http://dx.doi.org/10.3390/s18124375, URL: https://www.mdpi.com/1424-8220/18/12/4375.

Singh, S. H., van Breugel, F., Rao, R. P., et al. (2023). Emergent behaviour and neural dynamics in artificial agents tracking odour plumes. *Nature Machine Intelligence*, *5*, 58–70. http://dx.doi.org/10.1038/s42256-022-00599-w.

Song, C., He, Y., & Lei, X. (2019). Autonomous searching for a diffusive source based on minimizing the combination of entropy and potential energy. *Sensors*, *19*(11), http://dx.doi.org/10.3390/s19112465, URL: https://www.mdpi.com/1424-8220/19/11/2465.

Stockie, J. M. (2011). The mathematics of atmospheric dispersion modeling. *SIAM Review*, *53*(2), 349–372. http://dx.doi.org/10.1137/10080991X, arXiv:https://doi.org/10.1137/10080991X.

U. S. Environmental Protection Agency (2024). 2024 appendix w final rule — Guideline on air quality models. URL: https://www.epa.gov/scram/2024-appendix-w-final-rule. SCRAM: Guideline on Air Quality Models (Appendix W).

van Hasselt, H., Guez, A., & Silver, D. (2015). Deep reinforcement learning with double Q-learning. arXiv:1509.06461.

Veigele, W. J., & Head, J. H. (1978). Derivation of the Gaussian plume model. *Journal of the Air Pollution Control Association*, *28*(11), 1139–1140. http://dx.doi.org/10.1080/00022470.1978.10470720, arXiv:https://doi.org/10.1080/00022470.1978.10470720.

Vergassola, M., Villermaux, E., & Shraiman, B. I. (2007). 'Infotaxis' as a strategy for searching without gradients. *Nature*, *445*(7126), 406–409. http://dx.doi.org/10.1038/nature05464, URL: https://www.nature.com/articles/nature05464.

Wang, Z.-P., & Wu, H.-N. (2020). Gas source localization using improved multi-agent reinforcement learning. In *2020 Chinese automation congress* (pp. 6696–6701). http://dx.doi.org/10.1109/CAC51589.2020.9327850.

Yungaicela-Naula, N. M., Zhang, Y., Garza-Castañon, L. E., & Minchala, L. I. (2018). UAV-based air pollutant source localization using gradient and probabilistic methods. In *2018 international conference on unmanned aircraft systems* (pp. 702–707). http://dx.doi.org/10.1109/ICUAS.2018.8453430.

Zardi, D., Falocchi, M., Giovannini, L., Tirler, W., Tomasi, E., Antonacci, G., Ferrero, E., Alessandrini, S., Jimenez, P. A., Kosovic, B., & Delle Monache, L. (2021). The Bolzano tracer experiment (BTEX). *Bulletin of the American Meteorological Society*, *102*(5), E966–E989. http://dx.doi.org/10.1175/BAMS-D-19-0024.1.

Zhao, Y., Chen, B., Wang, X., Zhu, Z., Wang, Y., Cheng, G., Wang, R., Wang, R., He, M., & Liu, Y. (2022). A deep reinforcement learning based searching method for source localization. *Information Sciences*, *588*(C), 67–81. http://dx.doi.org/10.1016/j.ins.2021.12.041.

Zhu, P., Li, X., Poupart, P., & Miao, G. (2018). On improving deep reinforcement learning for POMDPs. arXiv:1704.07978. URL: https://arxiv.org/abs/1704.07978.